Architecture Decision-Making By Example

Andrew Harmel-Law
(he/him)
Tech Principal @ Thoughtworks
@al94781

What’s ahead of us?

(Plus a note on how we’ll learn together.)

Part One:

The importance of making your own architectural decisions

S1: Why are we here?
(5 minutes)

High-rise skyscraper building, Venus Formby, Public Domain.

Gedling Miners,
Public Domain
(via Guardian.co.uk)

Group Discussion:
What are your experiences of architectural decision-making and expectations for the next three hours?

Why are both these memes failing so badly?

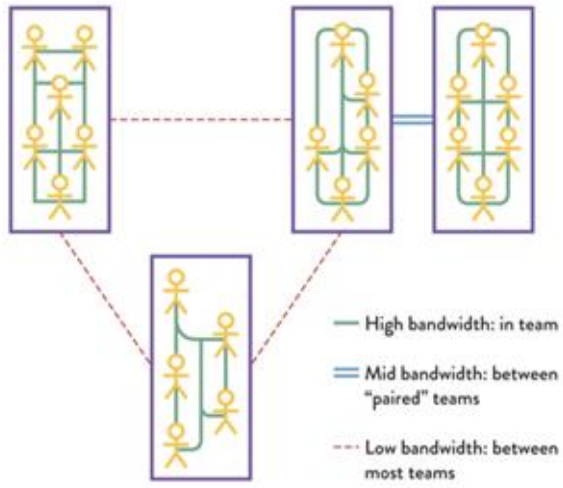


Figure 2.5: Inter-Team Communication

Communication within teams is high bandwidth. Communication between two "paired" teams can be mid bandwidth. Communication between most teams should be low bandwidth.

There are new circumstances:

- New ways of organising
- New ways of delivering & running software
- New runtimes
- New architectures

How we “do” architecture hasn’t kept up

- and so many teams aren’t coping

S1: Why are we here?
(Summary)

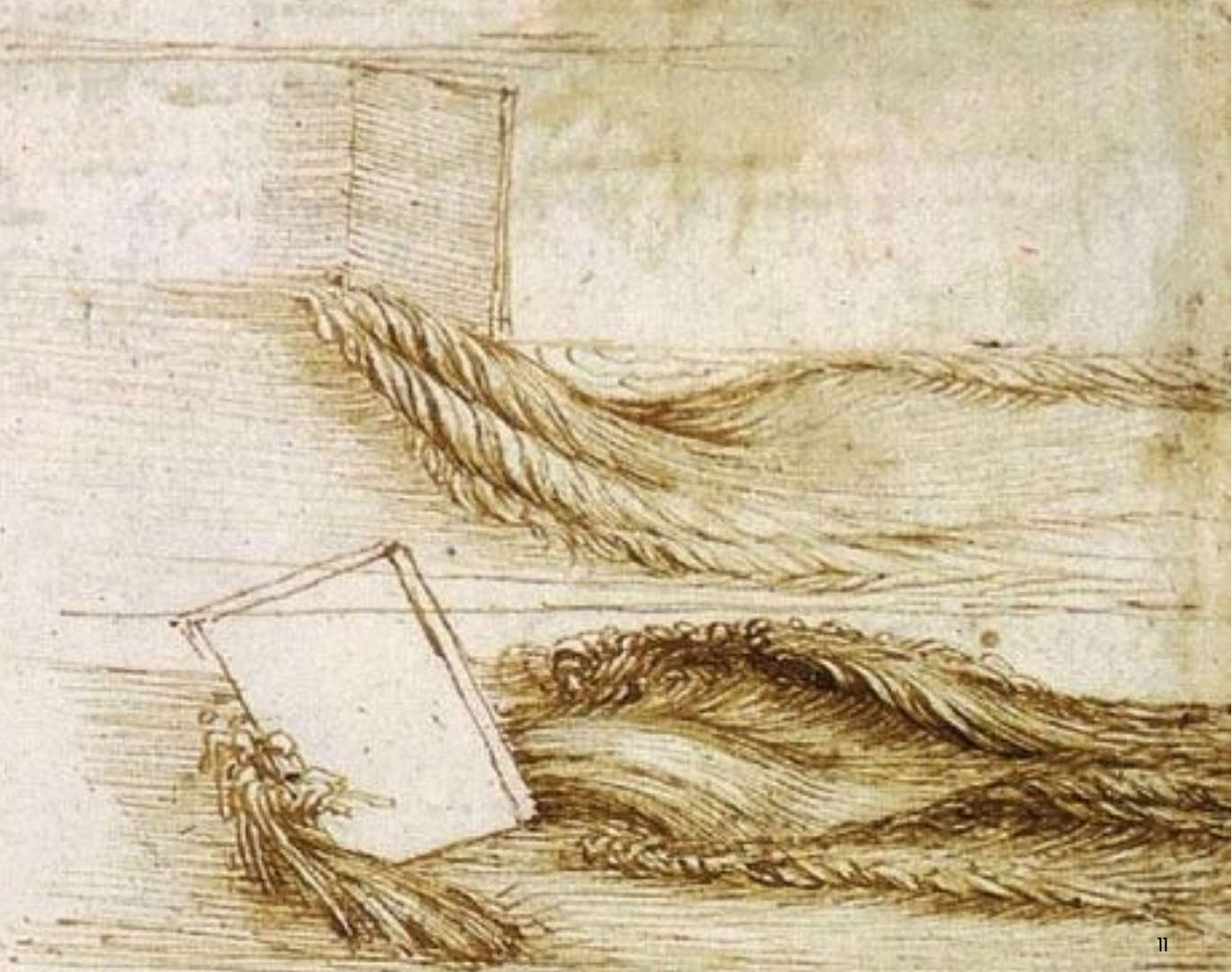
- The old approaches to “doing” architecture are failing
- Because the systems we build, & the ways we build them have moved on

- Any questions or thoughts to share?

S2: Why it’s important for developers to make architectural decisions

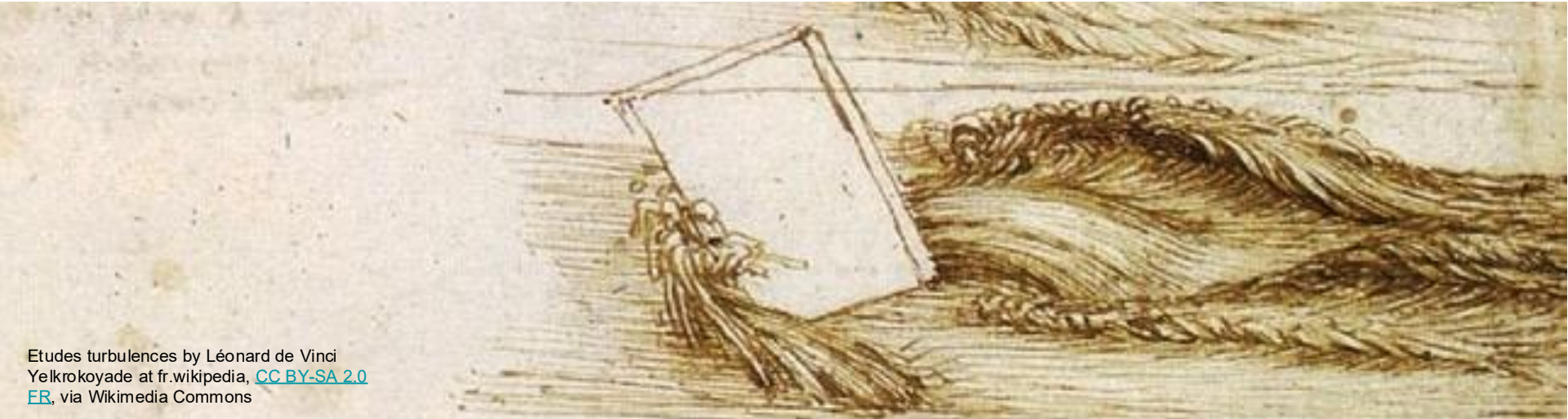
(15 minutes)

It’s always been about flow

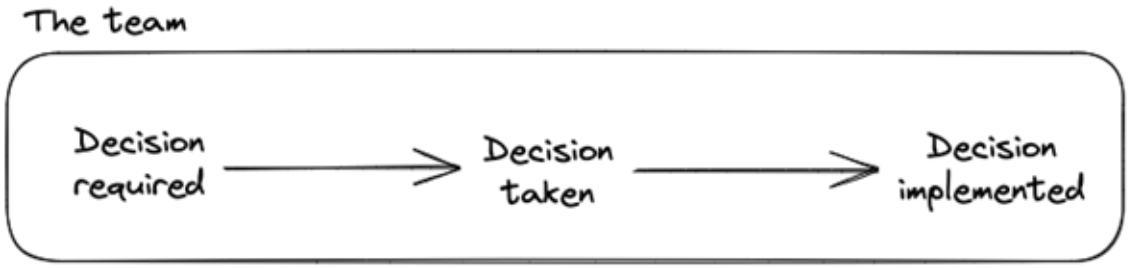


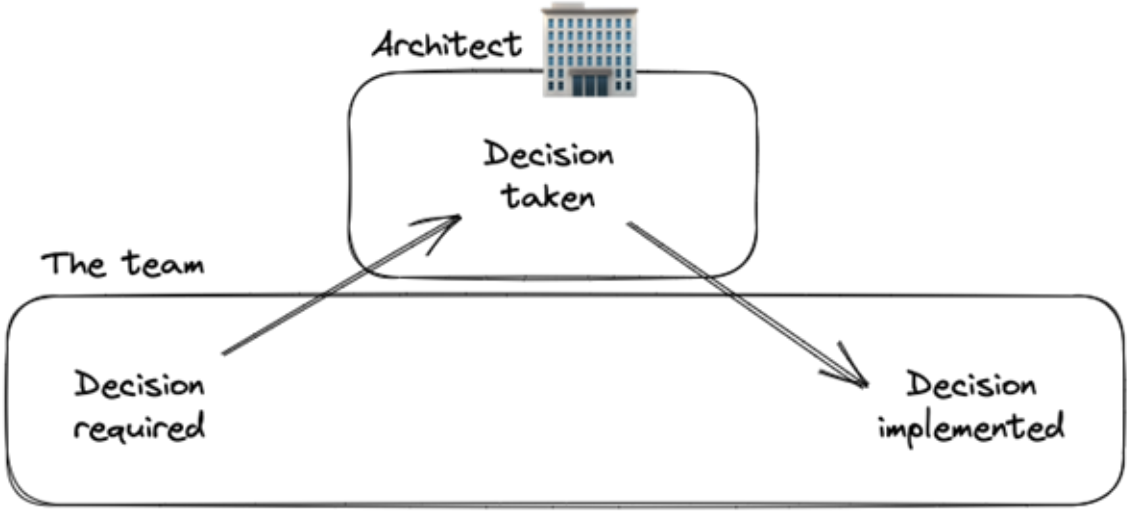
Etudes turbulences by Léonard de Vinci
Yelkrokoyade at fr.wikipedia, CC BY-SA 2.0 FR, via Wikimedia Commons

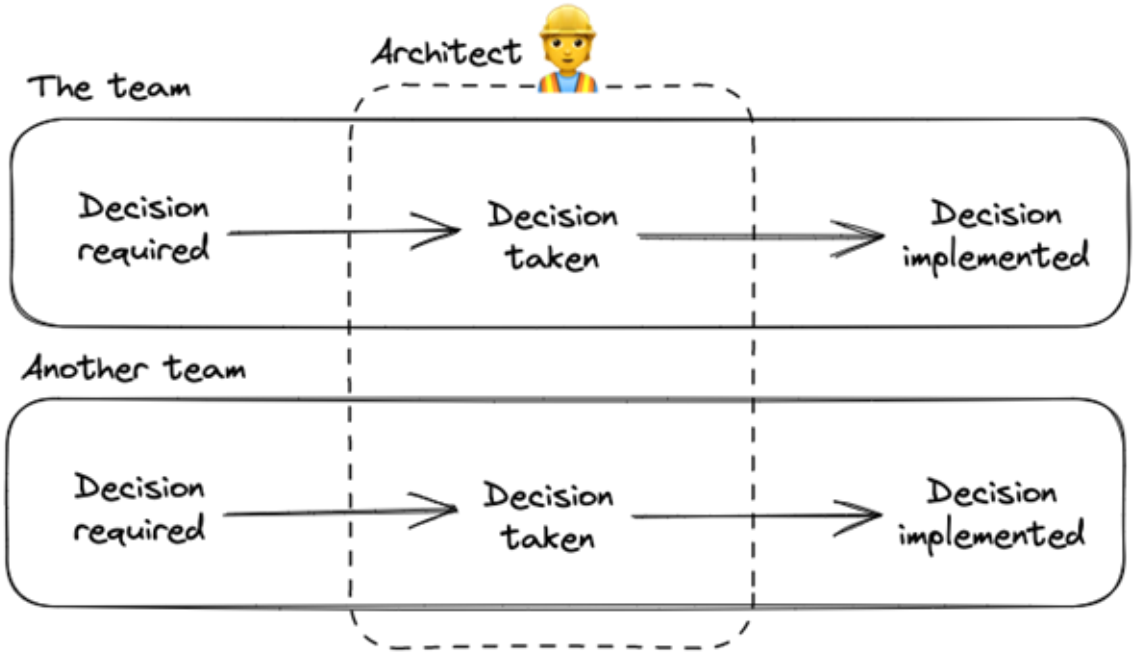
Group Discussion:
What are your experiences of self-management / autonomy and flow?



Etudes turbulences by Léonard de Vinci
Yelkrokoyade at fr.wikipedia, CC BY-SA 2.0 FR, via Wikimedia Commons


The team
Decision required
Decision taken
Decision implemented


Architect
Decision taken
The team
Decision required
Decision implemented


Architect
The team
Decision required
Decision taken
Decision implemented
Another team
Decision required
Decision taken
Decision implemented

S2: Why it's important for developers to make architectural decisions (Summary)

- Flow of change is one of the most important things in modern software development
- It has affected our architectures, how we deploy and run, and how we organise
- Now we need to change how we make decisions too

- Any questions or thoughts to share?

S3: Separating “architecting” from “Architects” (5 minutes)

Who makes architectural decisions?

Frank Lloyd Wright, Architect, Drafting the Marin County Civic Center, 1958 or 1959 by sswj
Attribution-NonCommercial-NoDerivs 2.0 Generic
(CC BY-NC-ND 2.0)

Group Discussion:
Why might <u>you</u> be the best person to make a decision?

What would you need to take this step?

What would you need to take this step?

- Permission

What would you need to take this step?

- Permission
- Training

What would you need to take this step?

- Permission
- Training
- Experience / support

What would you need to take this step?

- Permission
- Training
- Experience / support
- Access to things (human & non-human)

S3: Separating “architecting” from “Architects” (Summary)

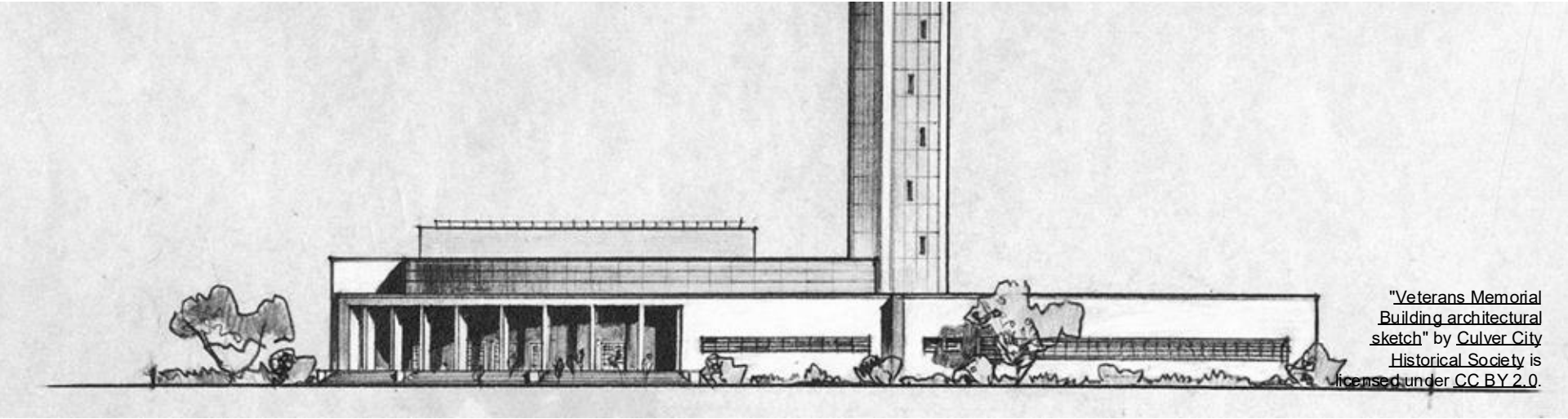
- “Architectural” decisions aren’t only decisions which are made by someone with “Architect” in their job title

- Any questions or thoughts to share?

S4: What is an “architectural decision”?

(5 minutes)

Group Discussion:
What kind of things are architectural decisions about?



"Veterans Memorial Building architectural sketch" by Culver City Historical Society is licensed under CC BY 2.0.

“those decisions which affect the structure,

It’s in the handout

— Michael Nygard
“Documenting Architecture Decisions” (2011)

“those decisions which affect the structure, non-functional characteristics,

It’s in the handout

— Michael Nygard
“Documenting Architecture Decisions” (2011)

“those decisions which affect the structure, non-functional characteristics, dependencies,

It’s in the handout

— Michael Nygard
“Documenting Architecture Decisions” (2011)

“those decisions which affect the structure, non-functional characteristics, dependencies, interfaces,

— Michael Nygard
“Documenting Architecture Decisions” (2011)

It’s in the handout

“those decisions which affect the structure, non-functional characteristics, dependencies, interfaces, or construction technology.”

— Michael Nygard

“Documenting Architecture Decisions” (2011)

It’s in the handout

S4: What is an “architectural decision”? (Summary)

- Architectural decisions can be big or small
- They can be made on purpose or by accident
- Nygard’s description is a nice way to decide if you have one

- Any questions or thoughts to share?

S5: Architectural Decision-Making Records (10 minutes)

What is an ADR?

ADR-01 Short Inventory Ids

ADR-01 Short Inventory Ids

Date: 2020-03-25
Driver: Wisen Tanasa

Status

Accepted - 2020-03-26

Decision

- We will increase the ID length when we're scaling up
- We're going with option 1, and will use nanoid
- with the following configuration:
  - Building ID:
    - ID Length: 6

Contex

Our building

[redacted]

We would like to make our building id shorter to make the seekers' UX better:

- They would be able to type the ID in the browser if they are printing the building page
- They would be able to share the URL without URL shortener

These are the decision criteria:

- Short
- Low collision probability (https://zelark.github.io/nano-id-cc/)

- Unambiguity, 0 (zero) and O (capital o) must not be confused
- Cost of implementation

Based on the above consideration, we have the following options:

1. AKVHAOFG: Random generated letters and numbers
2. 123456: Automatically generated sequence ID
3. canvas: Manually generated ID
4. 888882221: Pretty generated letters and numbers
5. canvas-AKVH: Combination of building name and generated id

On a discussion and research:

- We ruled out option 2, as it will be too costly to implement. It requires new infrastructure to be provisioned as we're adopting serverless architecture. Generated sequence is a concept from RDBMS. If we randomly generate numbers, the collision probability is also quite high and will need a longer id.
- We ruled out option 3, as it will involve too much human intervention
- We ruled out option 4, as we couldn't find a library that will easily support this
- We ruled out option 5, as we can use a slug if we're concerned about the URL

Consequences

- We may produce profanity words, but we're not bothered as we might get more traffic from the joke
- We don't have to provision any infrastructure, as the ID can be generated with nanoid tiny library
- If we generate 1 building ID per hour, we have 1% probability of 1 collision. We think this is quite high, and we don't think that we'll have 1 building ID per hour.

Advice

- Have we thought about profanity in the auto-generated text? (A considerate Product Manager)
- What's the possibility of id collisions? (The DBA)
- Have we looked at generating this in the database? (Also the DBA)
- Does making it human-meaningful matter, or just human-readable? (Someone from UX)
- Can we list out all the places where we'll need to use this id? (Concerned Tech Lead from another team)
- Will the ids be exposed publicly? Have we thought about leakage of internal data models? (The Infosec Professional)

ADR-01 Short Inventory Ids

Date: 2020-03-25
Driver: Wisen Tanasa

Status

Accepted - 2020-03-25

Decision

- We will increase the ID length when we're scaling up
- We're going with option 1, and will use nanoid
- with the following configuration:
  - Building ID:
    - ID Length: 6
    - Characters: 23456789ABCDEFGHJKMNPQRSTUVWXYZ
  - Space ID:
    - ID Length: 8
    - Characters: 0123456789ABCDEFGHJKLMNOPQRSTUVWXYZabcdefghjkmnpqrstuvwxyz
  - Provider ID:
    - ID Length: 5
    - Characters: 0123456789ABCDEFGHJKLMNOPQRSTUVWXYZabcdefghjkmnpqrstuvwxyz

Context

Our building id is currently generated from a UUI

```
21ce8c84-33e5-4bae-a701-7e1a3a1
```

We would like to make our building id shorter to r

- They would be able to type the ID in the b
- They would be able to share the URL with

These are the decision criteria:

- Short
- Low collision probability (https://www.ph

- Unambiguity, 0 (zero) and O (capital o) m
- Cost of implementation

Based on the above consideration, we have the following options:

1. AKVHAOFG: Random generated letters and numbers
2. 123456: Automatically generated sequence ID
3. canvas: Manually generated ID
4. 888882221: Pretty generated letters and numbers
5. canvas-AKVH: Combination of building name and generated id

On a discussion and research:

- We ruled out option 2, as it will be too costly to implement. It requires new infrastructure to be provisioned as we're adopting serverless architecture. Generated sequence is a concept from RDBMS. If we randomly generate numbers, the collision probability is also quite high and will need a longer id.
- We ruled out option 3, as it will involve too much human intervention.
- We ruled out option 4, as we couldn't find a library that will easily support this.
- We ruled out option 5, as we can use a slug if we're concerned about the URL.

Consequences

- We may produce profanity words, but we're not bothered as we might get more traffic from the joke.
- We don't have to provision any infrastructure, as the ID can be generated with nanoid tiny library.
- If we generate 1 building ID per hour, we have 1% probability of 1 collision. We think this is quite high, and we don't think that we'll have 1 building ID per hour.

Advice

- Have we thought about profanity in the auto-generated text? (A considerate Product Manager)
- What's the possibility of id collisions? (The DBA)
- Have we looked at generating this in the database? (Also the DBA)
- Does making it human-meaningful matter, or just human-readable? (Someone from UX)
- Can we list out all the places where we'll need to use this id? (Concerned Tech Lead from another team)
- Will the ids be exposed publicly? Have we thought about leakage of internal data models? (The Infosec Professional)

Decision

- We will increase the ID length when we're scaling up
- We're going with random generated letters and numbers (option 1) and will use nanoid for this
- with the following configuration:
  - Building ID:

ADR-01 Short Inventory Ids

Date: 2020-03-25
Driver: Wisen Tanasa

Status

Accepted - 2020-03-25

Decision

- We will increase the ID length when we're scaling up
- We're going with option 1, and will use nanoid
- with the following configuration:
  - Building ID:
    - ID Length: 6
    - Characters: 23456789ABCDEFGHJKMNPQRSTUVWXYZ
  - Space ID:
    - ID Length: 8
    - Characters: 0123456789ABCDEFGHJKLMNOPQRSTUVWXYZabcdefghjkmnpqrstuvwxyz
  - Provider ID:
    - ID Length: 5
    - Characters: 0123456789ABCDEFGHJKLMNOPQRSTUVWXYZabcdefghjkmnpqrstuvwxyz

Context

Our building id is currently generated from a UUID, and looks like:

```
22cadcb6-00e5-4baa-a701-785854fc2a9e.
```

We would like to make our building id shorter to make the seekers' UX better:

- They would be able to type the ID in the browser if they are printing the building page
- They would be able to share the URL with

These are the decision criteria:

- Short
- Low collision probability (https://www.pill

- Unambiguity, 0 (zero) and O (capital o) m
- Cost of implementation

Based on the above consideration, we have the fo

1. AKVHAOFG: Random generated letters a
2. 123456: Automatically generated sequenc
3. canvas: Manually generated ID
4. 888882221: Pretty generated letters and n
5. canvas-AKVH: Combination of building na

On a discussion and research:

- We ruled out option 2, as it will be too cost to be provisioned as we're adopting servic concept from ROBINS. If we randomly ge quite high and will need a longer id.
- We ruled out option 3, as it will involve too much human intervention
- We ruled out option 4, as we couldn't find a library that will easily support this
- We ruled out option 5, as we can use a slug if we're concerned about the URL

Consequences

- We may produce profanity words, but we're not bothered as we might get more traffic from the joke
- We don't have to provision any infrastructure, as the ID can be generated with nanoid tiny library
- If we generate 1 building ID per hour, we have 1% probability of 1 collision. We think this is quite high, and we don't think that we'll have 1 building ID per hour.

Advice

- Have we thought about profanity in the auto-generated text? (A considerate Product Manager)
- What's the possibility of id collisions? (The DBA)
- Have we looked at generating this in the database? (Also the DBA)
- Does making it human-meaningful matter, or just human-readable? (Someone from UX)
- Can we list out all the places where we'll need to use this id? (Concerned Tech Lead from another team)
- Will the ids be exposed publicly? Have we thought about leakage of internal data models? (The Infosec Professional)

Context

Our building id is currently generated from a UUID, and looks like:

```
22cadcb6-00e5-4baa-a701-785854fc2a9e.
```

We would like to make our building id shorter to make the seekers' UX better:

- They would be able to type the ID in the browser if they are printing the building page

ADR-01 Short Inventory Ids

Date: 2020-03-25
Driver: Wisen Tanasa

Status

Accepted - 2020-03-26

Decision

- We will increase the ID length when we're scaling up
- We're going with option 1, and will use nanoid
- with the following configuration:
  - Building ID:
    - ID Length: 6
    - Characters: 23456789ABCDEFGHJKMNPQRSTUVWXYZ
  - Space ID:
    - ID Length: 8
    - Characters: 0123456789ABCDEFGHJKLMNOPQRSTUVWXYZabcdefghjkmnpqrstuvwxyz
  - Provider ID:
    - ID Length: 5
    - Characters: 0123456789ABCDEFGHJKLMNOPQRSTUVWXYZabcdefghjkmnpqrstuvwxyz

Context

Our building id is currently generated from a UUID, and looks like:

```
21ceb0b4-31e5-4bae-a701-74f4b1efc2a9e
```

We would like to make our building id shorter to make the seekers' UX better:

- They would be able to type the ID in the browser if they are printing the building page
- They would be able to share the URL without URL shortener

These are the decision criteria:

- Short
- Low collision probability (https://sizeark.gitlab-

Options Considered

1. A4VHAOFG: Random generated letters and numbers
2. 123456: Automatically generated sequence ID:
3. canvas: Manually generated ID
4. BBBB2221: Pretty generated letters and numbers
5. canvas-A4VH: Combination of building name and generated Id

- Unambiguity, 0 (zero) and O (capital o) mu
- Cost of implementation

Based on the above consideration, we have the fo

1. A4VHAOFG: Random generated letters an
2. 123456: Automatically generated sequenc
3. canvas: Manually generated ID
4. BBBB2221: Pretty generated letters and n
5. canvas-A4VH: Combination of building na

On a discussion and research:

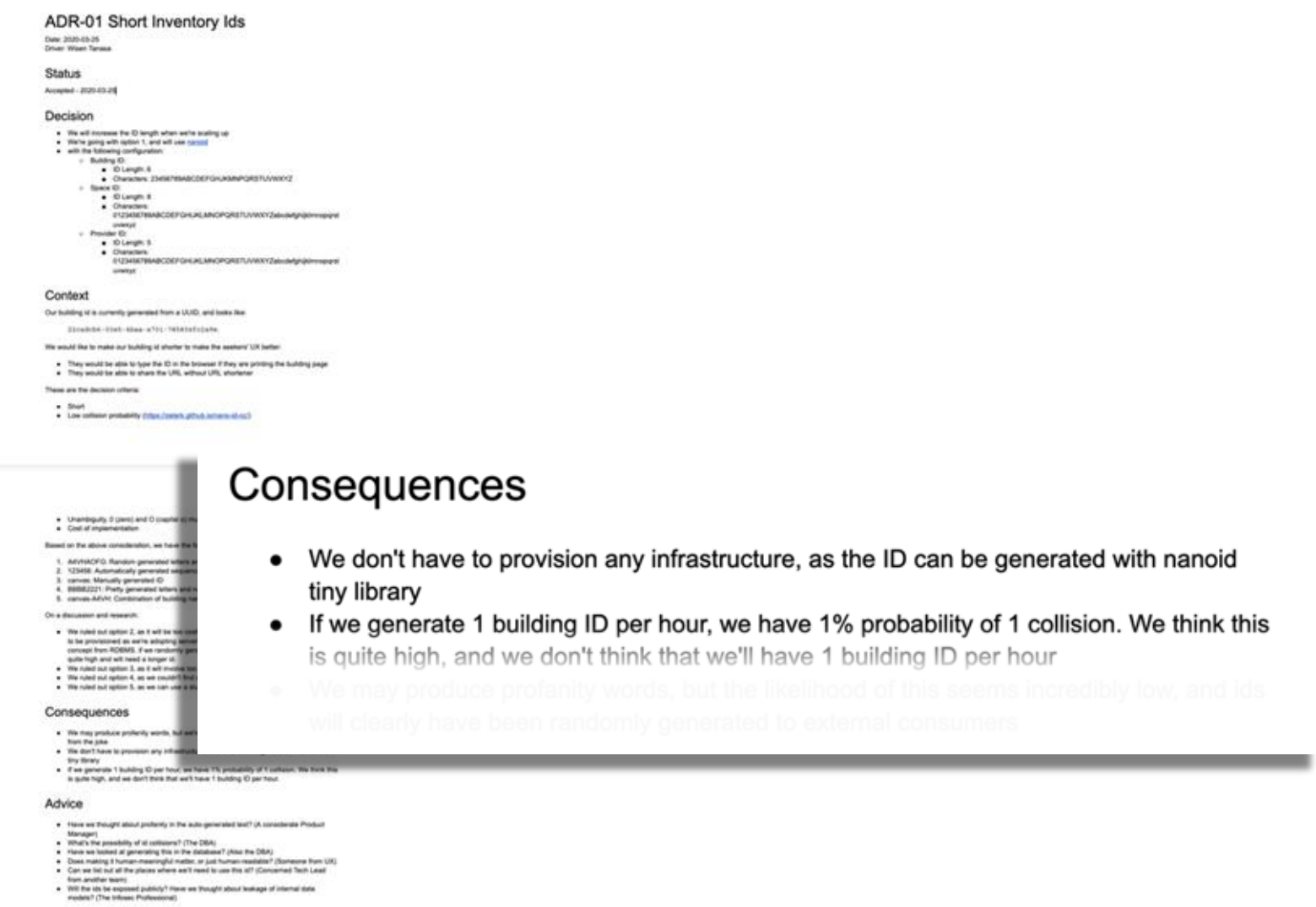
- We ruled out option 2, as it will be too cost
  to be provisioned as we're adopting serv
  concept from RDBMS. If we randomly gen
  quite high and will need a longer id.
- We ruled out option 3, as it will involve too
- We ruled out option 4, as we couldn't find a
- We ruled out option 5, as we can use a sh

Consequences

- We may produce profanity words, but we're not bothered as we might get more traffic from the jokes
- We don't have to provision any infrastructure, as the ID can be generated with nanoid tiny library
- If we generate 1 building ID per hour, we have 1% probability of 1 collision. We think this is quite high, and we don't think that we'll have 1 building ID per hour.

Advice

- Have we thought about profanity in the auto-generated text? (A considerate Product Manager)
- What's the possibility of id collisions? (The DBA)
- Have we looked at generating this in the database? (Also the DBA)
- Does making it human-meaningful matter, or just human-readable? (Someone from UX)
- Can we list out all the places where we'll need to use this id? (Concerned Tech Lead from another team)
- Will the ids be exposed publicly? Have we thought about leakage of internal data models? (The Infosec Professional)



ADR-01 Short Inventory Ids

Date: 2020-03-25
Driver: Wisen Tanasa

Status

Accepted - 2020-03-25

Decision

- We will increase the ID length when we're scaling up
- We're going with option 1, and will use nanoid
- with the following configuration:
  - Building ID:
    - ID Length: 6
    - Characters: 23456789ABCDEFGHJKMNPQRSTUVWXYZ
  - Space ID:
    - ID Length: 8
    - Characters: 0123456789ABCDEFGHJKLMNOPQRSTUVWXYZabcdefghjkmnpqrstuvwxyz
  - Provider ID:
    - ID Length: 5
    - Characters: 0123456789ABCDEFGHJKLMNOPQRSTUVWXYZabcdefghjkmnpqrstuvwxyz

Context

Our building id is currently generated from a UUID, and looks like:

```
21ceb0b4-31e5-4bae-a7f1-74f4b1efc2a9e
```

We would like to make our building id shorter to make the seekers' UX better:

- They would be able to type the ID in the browser if they are printing the building page
- They would be able to share the URL without URL shortener

These are the decision criteria:

- Short
- Low collision probability (https://celerk.github.io/nano-id-cc/)

- Unambiguity, 0 (zero) and O (capital o) m
- Cost of implementation

Based on the above consideration, we have the fo

1. AKVHAOFG: Random generated letters a
2. 123456: Automatically generated sequenc
3. canvas: Manually generated ID
4. 6666b2221: Pretty generated letters and n
5. canvas-AKVH: Combination of building na

On a discussion and research:

- We ruled out option 2, as it will be too cost to be provisioned as we're adopting serve concept from RDBMS. If we randomly gen quite high and will need a longer id.
- We ruled out option 3, as it will involve too
- We ruled out option 4, as we couldn't find
- We ruled out option 5, as we can use a sh

Consequences

- We may produce profanity words, but we from the jokes
- We don't have to provision any infrastruc tiny library
- If we generate 1 building ID per hour, we have 1% probability of 1 collision. We think this is quite high, and we don't think that we'll have 1 building ID per hour.

Advice

- Have we thought about profanity in the auto-generated text? (A considerate Product Manager)
- What's the possibility of id collisions? (The DBA)
- Have we looked at generating this in the database? (Also the DBA)
- Does making it human-meaningful matter, or just human-readable? (Someone from UX)
- Can we list out all the places where we'll need to use this id? (Concerned Tech Lead from another team)
- Will the ids be exposed publicly? Have we thought about leakage of internal data models? (The Infosec Professional)

Consequences

- We don't have to provision any infrastructure, as the ID can be generated with nanoid tiny library
- If we generate 1 building ID per hour, we have 1% probability of 1 collision. We think this is quite high, and we don't think that we'll have 1 building ID per hour
- We may produce profanity words, but the likelihood of this seems incredibly low, and ids will clearly have been randomly generated to external consumers

Michael Nygard
@mtnygard

These are good examples of ADRs. Succinct and written in declarative, fact-based style. Not trying to "sell" the decision, just saying what it is.

It's in the handout

ADR Key Points:

- Lightweight
- Point-in-time (you don't update them once the decision is made - just add another)
- They're a tool to help make decisions, not just a record of them

S5: Architectural Decision-Making Records (Summary)

- ADRs are a great help when making a decision
- All parts have a key value
- They're point-in-time
- They're small

- Any questions or thoughts to share?

S6: Let’s make our decision (20 minutes)

It’s in the handout

Exercise 1:
Making our decision with ADRs
Scenario (it's in your handout too)

It's in the handout

- We **already do CI/CD** - when we merge to main, a pipeline takes our code, via a series of stages, to production

Exercise 1:
Making a decision with ADRs
Scenario

It's in the handout

- We **already do CI/CD** - when we merge to main, a pipeline takes our code, via a series of stages, to production
- But this means we currently have **long-lived branches** as we wait for a feature to be "complete" so it can go to prod

Exercise 1:
Making a decision with ADRs
Scenario

It's in the handout

- We **already do CI/CD** - when we merge to main, a pipeline takes our code, via a series of stages, to production
- But this means we currently have **long-lived branches** as we wait for a feature to be "complete" so it can go to prod
- These branches are causing us real **merge pain**, and so we want to do away with them and push straight to main - we want to **adopt trunk-based development** as a team dev practice

Exercise 1:
Making a decision with ADRs
Scenario

It's in the handout

- We **already do CI/CD** - when we merge to main, a pipeline takes our code, via a series of stages, to production
- But this means we currently have **long-lived branches** as we wait for a feature to be "complete" so it can go to prod
- These branches are causing us real **merge pain**, and so we want to do away with them and push straight to main - we want to **adopt trunk-based development** as a team dev practice
- But that means we'll need a way to **deploy code to production which isn't "done"** (beyond compiling and passing all the tests)

Exercise 1:
Making a decision with ADRs
Scenario

It's in the handout

- We **already do CI/CD** - when we merge to main, a pipeline takes our code, via a series of stages, to production
- But this means we currently have **long-lived branches** as we wait for a feature to be "complete" so it can go to prod
- These branches are causing us real **merge pain**, and so we want to do away with them and push straight to main - we want to **adopt trunk-based development** as a team dev practice
- But that means we'll need a way to **deploy code to production which isn't "done"** (beyond compiling and passing all the tests)
- The way to do this is with **release toggles** - a mechanism which allows teams to separate "deployment" (the code is deployed in an environment) from "release"

Exercise 1:
Making a decision with ADRs
Scenario

It's in the handout

- We **already do CI/CD** - when we merge to main, a pipeline takes our code, via a series of stages, to production
- But this means we currently have **long-lived branches** as we wait for a feature to be "complete" so it can go to prod
- These branches are causing us real **merge pain**, and so we want to do away with them and push straight to main - we want to **adopt trunk-based development** as a team dev practice
- But that means we'll need a way to **deploy code to production which isn't "done"** (beyond compiling and passing all the tests)
- The way to do this is with **release toggles** - a mechanism which allows teams to separate "deployment" (the code is deployed in an environment) from "release"
- We need to **decide what architectural form** our toggles will take

Exercise 1:
Making a decision with ADRs
Steps

It's in the handout

- Exercise 1.1 - Capture notes about your context (need, surrounding circumstances, history, etc)

Exercise 1:
Making a decision with ADRs
Steps

It's in the handout

- Exercise 1.1 - Capture notes about your context (need, surrounding circumstances, history, etc)
- Exercise 1.2 - Collate your options (how might you do this / not do this)

Exercise 1:
Making a decision with ADRs
Steps

It's in the handout

- Exercise 1.1 - Capture notes about your context (need, surrounding circumstances, history, etc)
- Exercise 1.2 - Collate your options (how might you do this / not do this)
- Exercise 1.3 - Think about the consequences of each of your options

S6: Let’s make a decision (Summary)

- The work that goes into writing your ADR (especially for others) makes your decision better

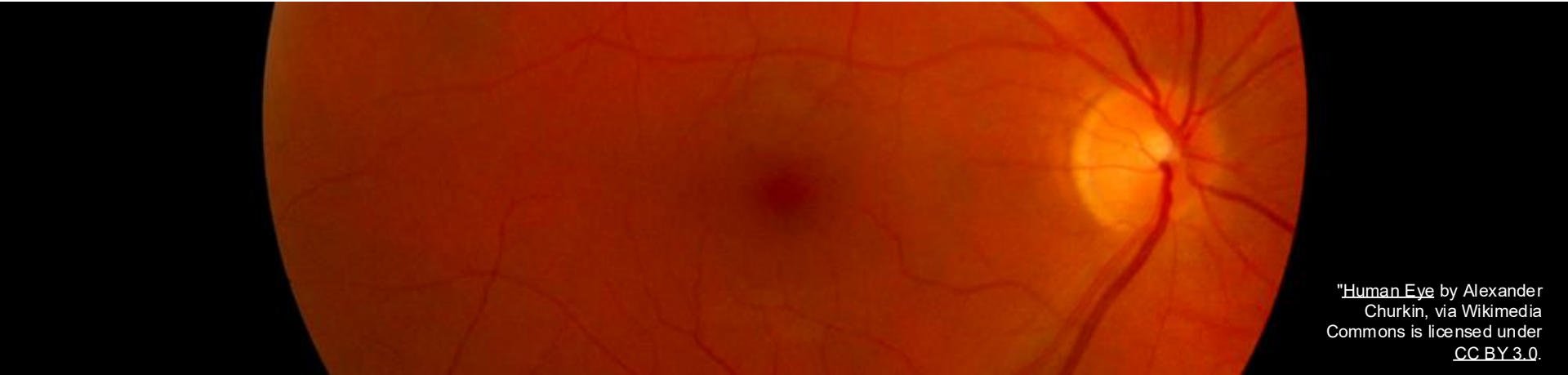
- Any questions or thoughts to share?

Part Two:

Leveling-up your decisions with the power of advice

S7: Decision blind-spots & how to combat them (20 minutes)

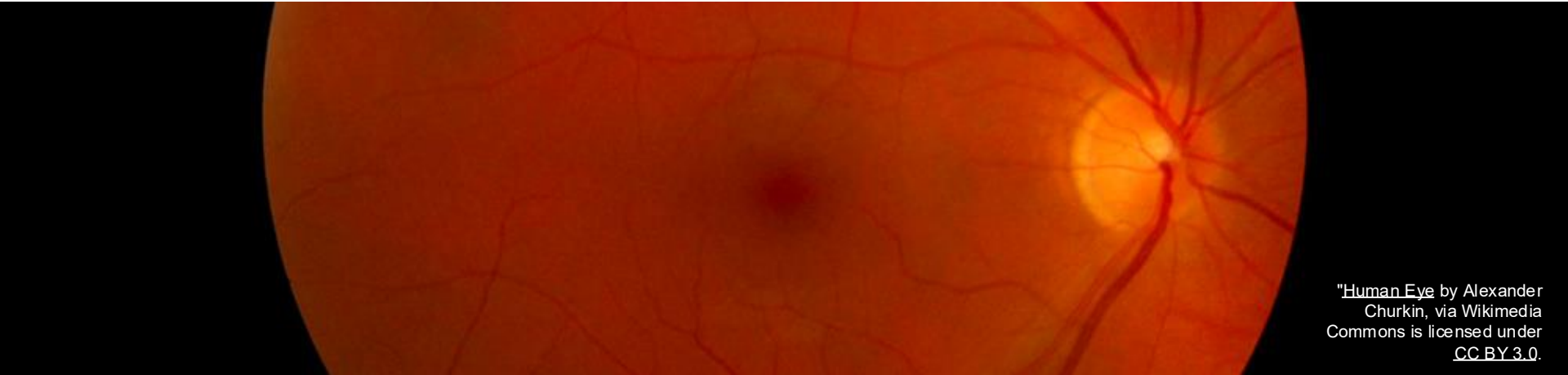
Group Discussion:
How might your peers in other teams feel about your decision?



"Human Eye by Alexander Churkin, via Wikimedia Commons is licensed under CC BY 3.0.

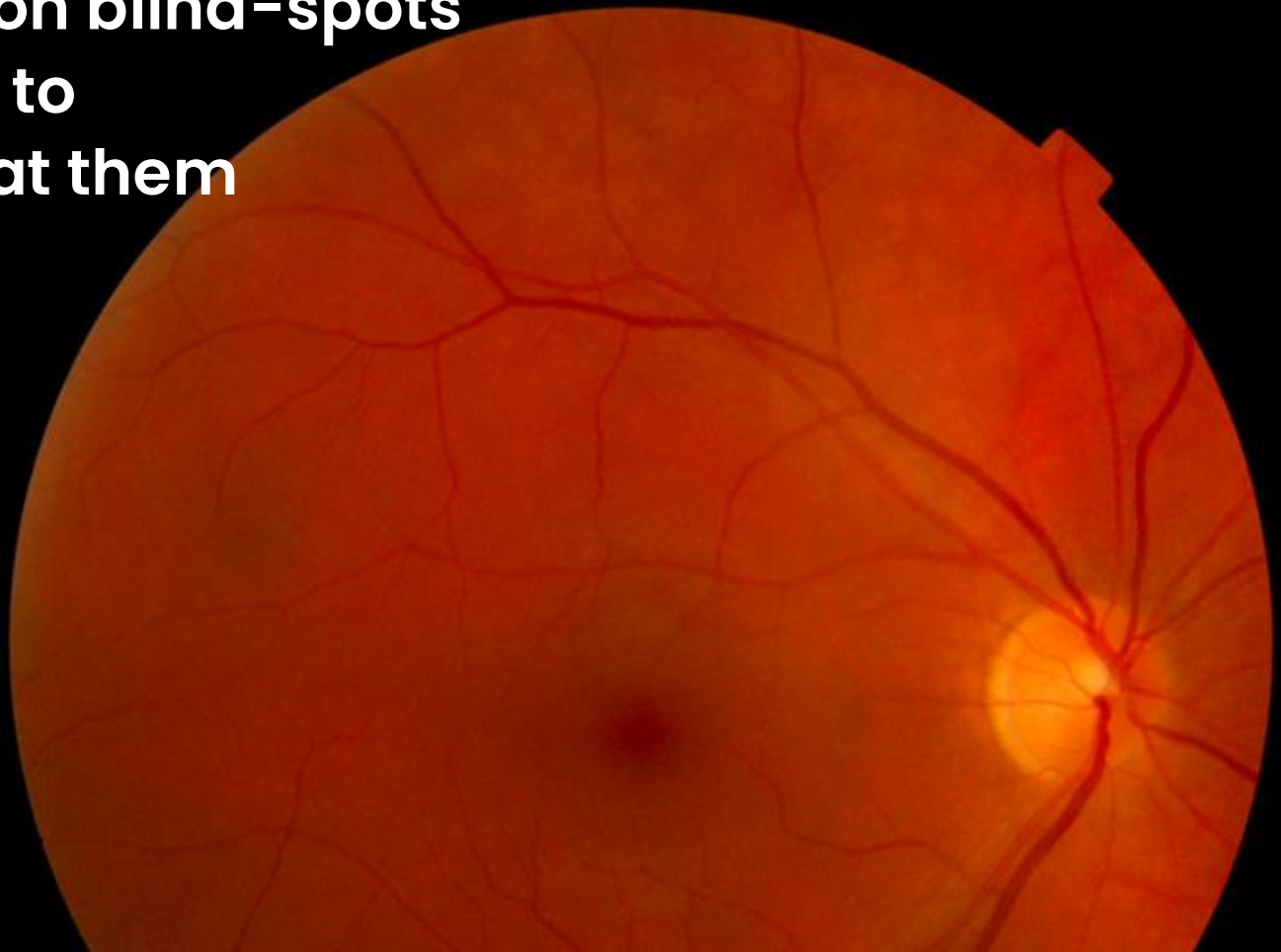
Group Discussion:
How might your peers in other teams feel about your decision?

What about those in other roles?



"Human Eye by Alexander Churkin, via Wikimedia Commons is licensed under CC BY 3.0.

Decision blind-spots & how to combat them



"Human Eye by Alexander Churkin, via Wikimedia Commons is licensed under CC BY 3.0.

It's in the handout

The Architectural Advice Process:

Anyone can make any decision.

It's in the handout

The Architectural Advice Process:

Anyone can make any decision.
But before doing so they *must seek:*

It's in the handout

The Architectural Advice Process:

Anyone can make any decision.
But before doing so they *must seek:*
advice from all affected parties

It's in the handout

The Architectural Advice Process:

Anyone can make any decision.
But before doing so they *must seek:*
advice from all affected parties
and people with expertise in the
matter.

ADR-01 Short Inventory Ids

Date: 2020-03-25
Driver: Wisen Tanasa

Status

Accepted - 2020-03-25

Decision

- We will increase the ID length when we're scaling up
- We're going with option 1, and will use nanoid
- with the following configuration:
  - Building ID:
    - ID Length: 6
    - Characters: 23456789ABCDEFGHJKMNPQRSTUVWXYZ
  - Space ID:
    - ID Length: 8
    - Characters: 0123456789ABCDEFGHJKLMNOPQRSTUVWXYZabcdefghjkmnpqrstuvwxyz
  - Provider ID:
    - ID Length: 5
    - Characters: 0123456789ABCDEFGHJKLMNOPQRSTUVWXYZabcdefghjkmnpqrstuvwxyz

Context

Our building id is currently generated from a UUID, and looks like:

```
21ceb0b4-31e5-4bae-a7f1-74f4b1efc2a9e
```

We would like to make our building id shorter to make the seekers' UX better:

- They would be able to type the ID in the browser if they are printing the building page
- They would be able to share the URL without URL shortener

These are the decision criteria:

- Short
- Low collision probability (https://zelark.github.io/nano-id-cc/)

- Unambiguity, 0 (zero) and O (capital o) m
- Cost of implementation

Based on the above consideration, we have the fo

1. AKVHAOFG: Random generated letters a
2. 123456: Automatically generated sequenc
3. canvas: Manually generated ID
4. 888882221: Pretty generated letters and n
5. canvas-AKVH: Combination of building na

On a discussion and research:

- We ruled out option 2, as it will be too cost to be provisioned as we're adopting servic concept from RDBMS. If we randomly gen quite high and will need a longer id.
- We ruled out option 3, as it will involve too
- We ruled out option 4, as we couldn't find a
- We ruled out option 5, as we can use a sh

Consequences

- We may produce profanity words, but we'r from the joke
- We don't have to provision any infrastructu tiny library
- If we generate 1 building ID per hour, we is quite high, and we don't think that we'll

Advice

- Have we thought about profanity in the auto-generated text? (A considerate Product Manager)
- What's the possibility of id collisions? (The DBA)
- Have we looked at generating this in the database? (Also the DBA)
- Does making it human-meaningful matter, or just human-readable? (Someone from UX)
- Can we list out all the places where we'll need to use this id? (Concerned Tech Lead from another team)
- Will the ids be exposed publicly? Have we thought about leakage of internal data models? (The Infosec Professional)

Advice

- Have we thought about profanity in the auto-generated text? (A considerate Product Manager)
- What's the possibility of id collisions? (The DBA)
- Have we looked at generating this in the database? (Also the DBA)
- Does making it human-meaningful matter, or just human-readable? (Someone from UX)
- Can we list out all the places where we'll need to use this id? (Concerned Tech Lead from another team)

An example of advice-seeking

And the roles which people play in the process

Group Discussion:
Who might we want to seek advice from?

S7: Decision blind-spots & how to combat them (Summary)

- The advice process removes blind-spots
- Making your decisions better and raises your awareness

- Any questions or thoughts to share?

S8: Seeking (and offering) advice
(15 minutes)

Advice is:

"guidance or recommendations offered with regard to prudent future action."

Advice is:

"guidance or recommendations offered with regard to prudent future action."

It's NOT opinion-giving.

Exercise 2:
Seeking and offering advice on our ADR

It's in the handout

- Step 1 - set the ADR status to "Proposed"

Exercise 2:
Seeking and offering advice on our ADR

It's in the handout

- Step 1 - set the ADR status to "Proposed"
- Step 2 - think: who should we seek advice from?

Exercise 2:
Seeking and offering advice on our ADR

It's in the handout

- Step 1 - set the ADR status to "Proposed"
- Step 2 - think: who should we seek advice from?
- Step 3 - offer advice

Group Discussion:
Re-evaluating our ADR based on advice received

S8: Seeking (and offering) advice (Summary)

- Advice is not opinion-giving
- Advice says “why”
- Advice is an incredibly powerful tool when deciding

- Any questions or thoughts to share?

S9: Going meta - Advice about making architectural decisions

(10 minutes)

Group Discussion:
Any advice for the group on how to make great decisions?

Some decision-making power moves

It's in the handout

USE THE ADR:

CONTEXT - get this clear
OPTIONS - not just your preferences
BENEFITS / DRAWBACKS - ditto
THINK ABOUT THE READER

Some decision-making power moves

It's in the handout

THEN:	Think about the FUTURE SCOPE: Can you make it smaller? NFRs: Are there any hiding in there? *CONSIDER DIFFERENT PERSPECTIVES*
USE THE ADR:	CONTEXT - get this clear OPTIONS - not just your preferences BENEFITS / DRAWBACKS - ditto *THINK ABOUT THE READER*

Some decision-making power moves

It's in the handout

PROACTIVE ENGAGEMENT:	Three foundational skills: Metacognition - thinking about your thinking Respond rather than react Share the reasons for your advice
THEN:	Think about the FUTURE SCOPE: Can you make it smaller? NFRs: Are there any hiding in there? *CONSIDER DIFFERENT PERSPECTIVES*
USE THE ADR:	CONTEXT - get this clear OPTIONS - not just your preferences BENEFITS / DRAWBACKS - ditto *THINK ABOUT THE READER*

S9: Going meta - Advice about making architectural decisions (Summary)

- ADRs are a great foundation for good decisions
- Then think about the range of the decision (and reduce it)
- Engage proactively and think about your (and others) thinking

- Any questions or thoughts to share?

S10: Completing our ADR & taking our decision (10 minutes)

Exercise 3:
Taking the decision

It's in the handout

1. Updating the context, options, and consequences
2. Writing the decision summary
3. Updating the title
4. Updating the status

S10: Completing our ADR & taking our decision (Summary)

- Update the ADR as much as you feel appropriate based on the advice
- This is your decision - YOU are now accountable
- Make the title the only element you need to read

- Any questions or thoughts to share?

Part Three:

Why guess when you can learn?

S11: There are no “perfect decisions”
(15 minutes)

"Starlings over Gretna" by Walter Baxter. Licenced for reuse under CC Attribution-ShareAlike 2.0 Generic (CC BY-SA 2.0)

Group Discussion:
How might we get as close to “perfect” decisions as possible?

“Starlings over Gretna” by Walter Baxter. Licenced for reuse under CC Attribution-ShareAlike 2.0 Generic (CC BY-SA 2.0)

S11: There are no “perfect decisions” (Summary)

- Our systems are too complex to predict all the consequences of our decisions
- So stop trying
- Proceed by small, incremental steps and learn from the feedback

- Any questions or thoughts to share?

S12: Spikes - stop deciding, start experimenting (20 minutes)

What is a “Spike”? (and how can they lead to great ADRs?)

It’s in the handout

“S107E05100 - STS-107 - Chawla works at the MGM experiment in the Spacehab RDM during STS-107” Licenced for reuse under Public Domain Mark 1.0

SPIKE0001 - Can we have shorter, human-readable inventory ids?

Timebox

2 dev pairs for 3 days.

Context

Our current inventory ids across the company for buildings, spaces, providers, etc are generated from UUIDs. Sometimes humans have to read, share, or even manually enter these ids and it's painful due to their length and characters.

We want to investigate whether it's feasible to move to shorter, human-readable inventory ids.

Our decision criteria for selecting a solution is:

- Short
- Unambiguous to humans (0 (zero) and O (capital o) cannot be confused for example)
- URL-friendly
- Low collision probability
- Non-sequential
- Existing in-scope ids in all systems can be migrated to the new solution
- Works with our NodeJS / Typescript codebase
- Cost-effective to implement (do we need to pay for anything new?)
- Acceptable licence terms (certain OSS licences are fine, some are not)

Options considered

- Generate in our code
- Generate in our persistence stores
- Generate via a third-party tool / library we can run inside our network
  - E.g. Nanoid

Consequences

- Which teams will have systems affected, will be affected?
- Will any downstream teams be affected?
- Will InfoSec be concerned?
- Will this cost anything extra? Will this cost be one-off, annual, per transaction?
- Will this affect system performance?

ADR002 (Example) Short Inventory Ids

Date: 2020-03-25
Driver: Wisen Tanasa

Status

Accepted - 2020-03-25

Decision

- We will increase the ID length when we're scaling up
- We're going with random generated letters and numbers (option 1) and will use nanoid for this
- with the following configuration:
  - Building ID:
    - ID Length: 6
    - Characters: 23456789ABCDEFGHJKMNPQRSTUVWXYZ
  - Space ID:
    - ID Length: 8
    - Characters: 0123456789ABCDEFGHJKLMNOPQRSTUVWXYZabcdefghjkmnopqrstuvwxyz
  - Provider ID:
    - ID Length: 5
    - Characters: 0123456789ABCDEFGHJKLMNOPQRSTUVWXYZabcdefghjkmnopqrstuvwxyz

Context

Our building id is currently generated from a UUID, and looks like:

```
22ca8c0d-03e5-45aa-a791-745856f2ca9a
```

We would like to make our building id shorter to make the seekers' UX better:

- They can type the ID into the browser if they are printing the building page
- They can share the URL easily without a URL shortener

These are the decision criteria:

- Short
- Low collision probability (https://zelark.github.io/nano-id-cc/)
- Unambiguity, 0 (zero) and O (capital o) must not be confused
- Cost of implementation

Options Considered

1. AAVHAOFG: Random generated letters and numbers
2. 123456: Automatically generated sequence ID.
3. canvas: Manually generated ID
4. BBBB2221: Pretty generated letters and numbers
5. canvas-AAVH: Combination of building name and generated id

We then subjected these to discussion and research:

- We ruled out option 2, as it will be too costly to implement. It requires new infrastructure to be provisioned as we're adopting serverless architecture. Generated sequence is a concept from RDBMS. if we randomly generate numbers, the collision probability is also quite high and will need a longer id.
- We ruled out option 3, as it will involve too much human intervention
- We ruled out option 4, as we couldn't find a library that will easily support this.
- We ruled out option 5, as we can use a slug if we're concerned about the URL

Consequences

- We don't have to provision any infrastructure, as the ID can be generated with nanoid tiny library
- If we generate 1 building ID per hour, we have 1% probability of 1 collision. We think this

Exercise 4:
Let's write a Spike
Background

It's in the handout

- Investigate the best way to shorten inventory ids

OR

- Do you have an ADR you're thinking of writing as soon as you get off this call? Make it a spike first

Exercise 4:
Let’s write a Spike
Steps

It’s in the handout

- Step 1 - write the context
- Step 2 - brainstorm the options considered
- Step 3 - write questions to think about in the consequences
- Step 4 - write the title
- Step 5 - what’s the timebox?

S12: Spikes - stop deciding, start experimenting (Summary)

- Spikes are great for pre-decision experimentation and learning and they feed into ADRs perfectly

- Any questions or thoughts to share?

S13: How to start doing this today
(10 minutes)

Start now! (Adoption Patterns)

It's in the handout

Learn from what's around you

- Look back at decisions in your past (ones you made, or ones which others made for you)
- If it wasn't you, ask to hear more about the "making" part
- Consider this in the light of how things turned out (you can reconstruct the feedback loops)
  - Think specifically about how these decisions impacted flow; for you and for others
  - Then think about conceptual integrity; was it strengthened or weakened
  - Finally think about what made its way into PROD. How did that turn out?

Start now! (Adoption Patterns)

It's in the handout

Start small:

- Just doing it for yourself - run Spikes and write ADRs

Then scale up to colleagues in your team

Then inspire other teams to do the same

Keep track of (over time):

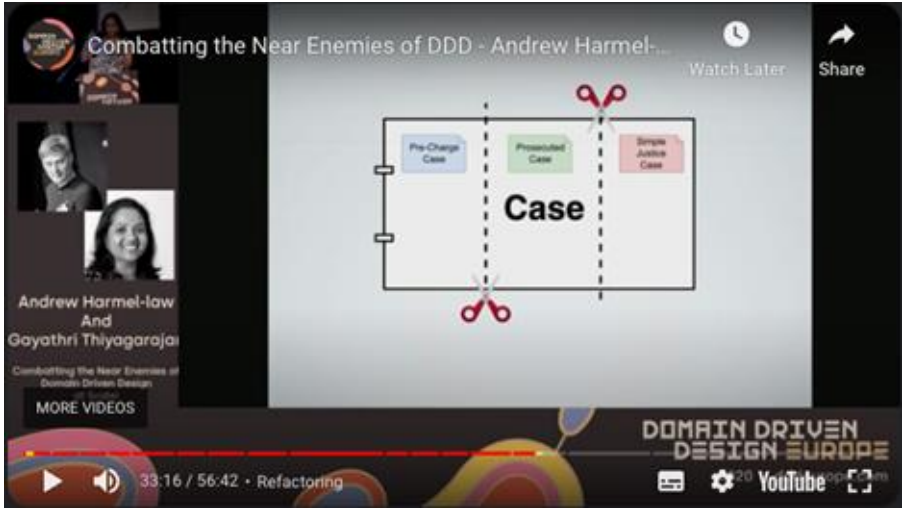
- Number of Spikes and ADRs
- Number of people / teams initiating them
- Time it takes to go from a Spike to an ADR being "Proposed", to it being "Accepted", and finally "Adopted"
- Number of people offering advice

Start now! (Adoption Patterns)

It's in the handout

Learn about others making decisions:

Read case studies. Gayathri Thiyagarajan and I talked about a series of decisions we made on a large programme of work for example (link in the handout.)



S13: How to start doing this today (Summary)

- Any questions or thoughts to share?

S14: Summing up
(10 minutes)

Summary (1/2)

- We saw how the best person to make and take a vast number of decisions is you
- We saw how it’s best to involve others for their advice
- We saw it’s better to let go and start with a Spike

Summary (2/2)

- We saw how ADRs great for making decisions
- We saw how broadening our horizons and being self-aware supercharges things
- We saw how getting comfortable being wrong, and learning as you go takes you even further

O'REILLY®

Thank you for your participation

What final questions do you have? (Or thoughts to share?)